Cupcake Project

Deltagere:
Navn / cph-email / GitHub brugernavn

Andreas Jensen - cph-aj645@stud.ek.dk - AndreasJensen1997

Morten Helander - cph-mh1358@stud.ek.dk - MortenHelander

Theis Rud Halkjær - cph-th463@stud.ek.dk - Tajan92

Alle elever i dette projekt er fra hold **B**

Dette projekt blev oprettet **Tirsdag D.7. april 2026** og afleveret **Fredag D. 17. april 2026**

Indholdsfortegnelse

Indhold

Cupcake Project 1
Indholdsfortegnelse 2
Indledning 3
Baggrund 3
Teknologi valg 3
Krav 4
Domæne model og ER diagram 5
Domænemodel 5
ER diagram 6
Navigationsdiagram 7
Særlige forhold 8
Status på implantation 9
Proces 10
Hvad var jeres planer for teamets arbejdsform og projektforløbet? 10
Hvad gik godt og hvad kunne have været bedre? 11
Hvad har I lært af processen og hvad vil I evt. gøre anderledes næste gang? 11

Indledning

Dette projekt omhandler en hjemmeside lavet til en fiktiv virksomhed, der skal sælge økologiske cupcakes fra Bornholm.
Projektet er lavet af tre datamatiker studerende på 2. semester fra EK Lyngby, målet med denne rapport er at kunne belyse andre datamatikere på samme niveau om indholdet af vores projekt.
I rapporten vil vi komme ind på, hvilke teknologier vi har brugt, samt diagrammer, hvilke krav kunden har, særlige forhold i vores program,
vores status på implantation, deri vores fejl, mangler og ting vi gerne ville have nået og vores arbejdsproces.

Baggrund

Virksomheden bag dette projekt er Olsker Cupcakes. Et småt iværksættereventyr med fokus på økologi fra Bornholm.
Firmaet havde et vision om at en kunde skulle kunne oprette en bruger i deres system, bestille cupcakes, samt betale for dem.
Alle cupcakes i firmaet består af en top og en bund og en kunde skulle kunne mix & match som de ønsker.
Udover det, ønskede de at brugeren kunne se sine tidligere ordre, samt en admin profil som har adgang til en oversigt over alle brugere og deres ordre. Som admin skulle man også kunne slette ordre og tilføje balance til en brugers konto.

Teknologi valg

IntelliJ j 2026.1

Postgres 42.7.5

Thymeleaf 3.1.2.RELEASE

Java sdk 21

html 5

Java Script ECMAScript 6+

css

lombok 1.18.44

Krav

Olsker Cupcakes havde et sæt user stories som de ønskede at vi fulgte.

De var således:

US-1: Som kunde kan jeg bestille og betale cupcakes med en valgfri bund og top, sådan at jeg senere kan køre forbi butikken i Olsker og hente min ordre.

US-2 Som kunde kan jeg oprette en konto/profil for at kunne betale og gemme en en ordre.

US-3: Som administrator kan jeg indsætte beløb på en kundes konto direkte i Postgres, så en kunde kan betale for sine ordrer.

US-4: Som kunde kan jeg se mine valgte ordrelinier i en indkøbskurv, så jeg kan se den samlede pris.

US-5: Som kunde eller administrator kan jeg logge på systemet med email og kodeord. Når jeg er logget på, skal jeg kunne se min email på hver side (evt. i topmenuen, som vist på mockup'en).

US-6: Som administrator kan jeg se alle ordrer i systemet, så jeg kan se hvad der er blevet bestilt.

US-7: Som administrator kan jeg se alle kunder i systemet og deres ordrer, sådan at jeg kan følge op på ordrer og holde styr på mine kunder.

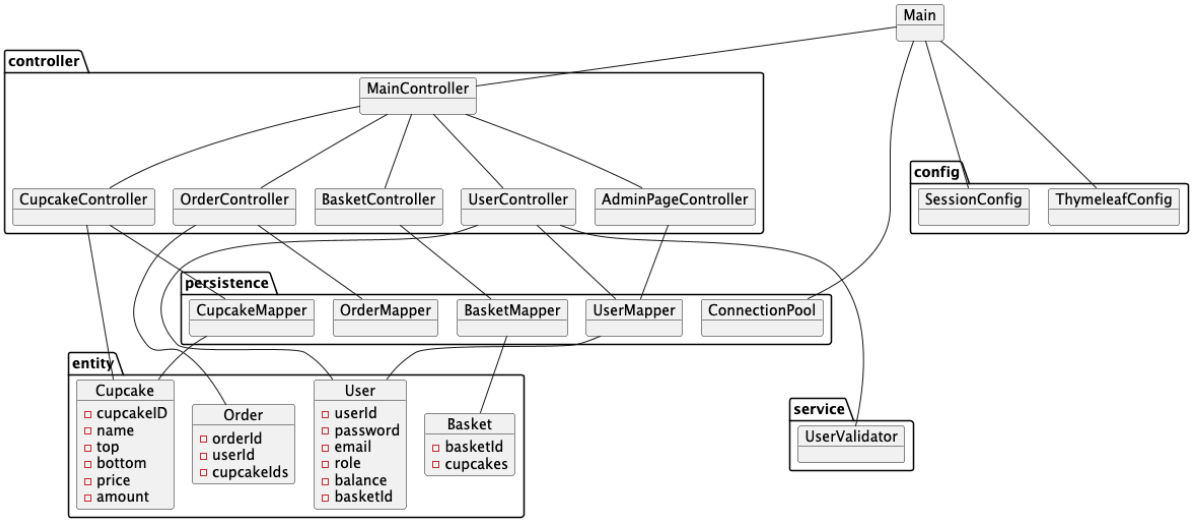
US-8: Som kunde kan jeg fjerne en ordrelinie fra min indkøbskurv, så jeg kan justere min ordre.

US-9: Som administrator kan jeg fjerne en ordre, så systemet ikke kommer til at indeholde udgyldige ordrer. F.eks. hvis kunden aldrig har betalt.

Domæne model og ER diagram

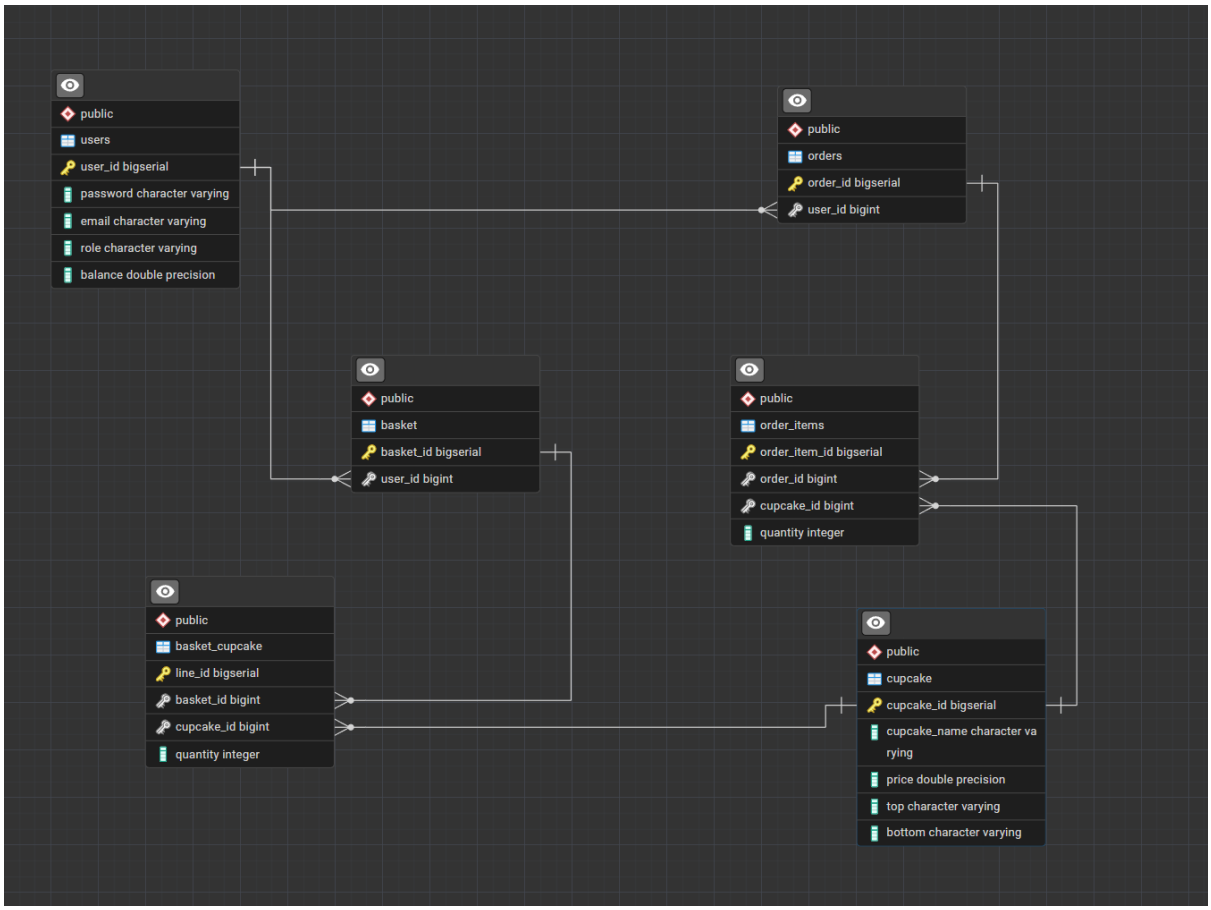
Domænemodel

Under opbygningen af domænemodellen brugte vi meget tid på hvilke entiteter vi skulle have med, specielt cupcake som vi først tænkte skulle deles op i top og bund. Vi endte dog ud med alt i en cupcake entitet, som gjorde bygningen meget enklere.
Basket blev også tilføjet senere, for at kunne oprette en kurv for hver bruger med flere cupcakes i.



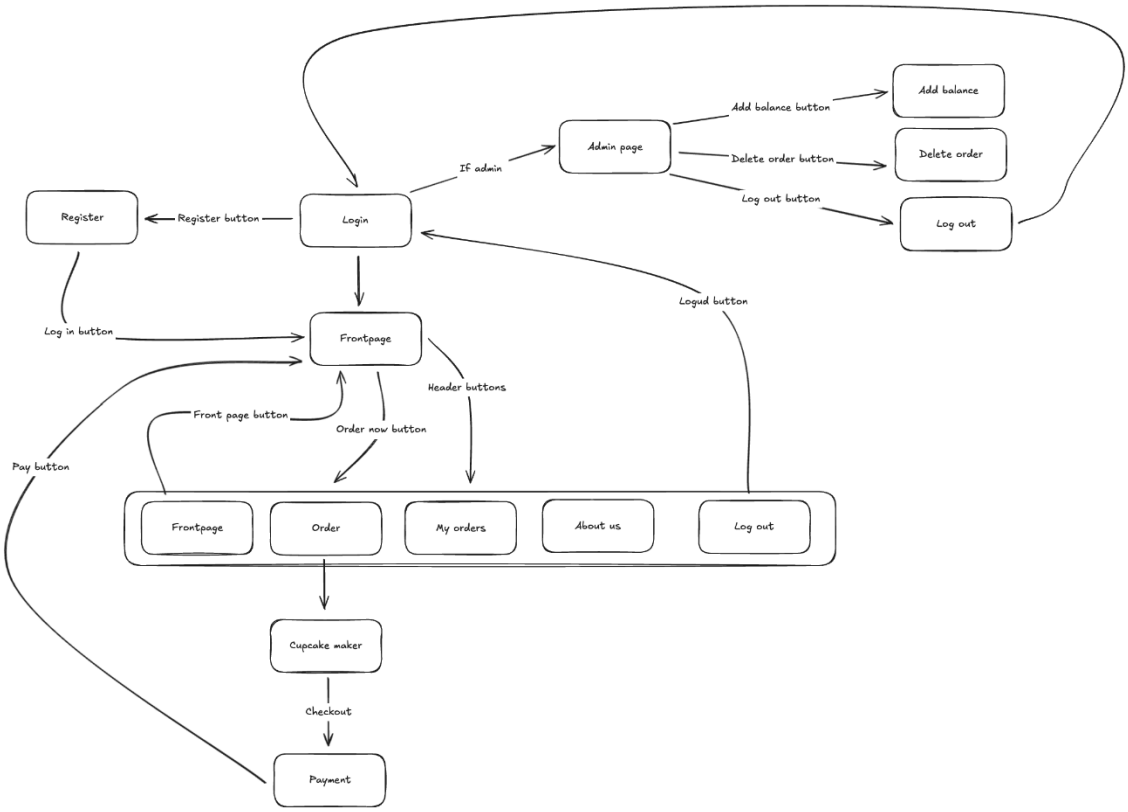
ER diagram

Vores ERD-diagram som vi lavede første dag, var et godt udgangspunkt, men i takt med vi kom længere med projektet, blev det tydeligt at vi måtte genoverveje vores Databases struktur.
Til at starte med havde tabellen "cupcakes" et "user_id", som en foreign key i tabellen "users". Dette viste sig ikke at være den bedste ide, da dette betød at der skulle oprettes en unik cupcake i databasen,
hver gang en bruger tilføjede en til sin kurv. Derudover var det ikke hensigtsmæssigt, at en cupcake skulle tilknyttes en og kun en bruger og det betød også at der kun kunne eksistere en cupcake per ordre.
Senere i projektet ændrede vi det, sådan så "cupcakes" kun indholdte information nødvendig for den. I stedet skabte vi den relationelle forbindelse ved hjælp af to "conjuction tables", "basket_cupcake" og "order_items",
som bruges til at knytte en cupcake til en ordre, hvor "order_items" tabellen så beskriver mængden af cupcakes.



Navigationsdiagram

I vores navigations diagram har vi to "system states". Man kan enten være logget ind som bruger eller som administrator. En normal bruger har ikke adgang til admins sider og en admin har ikke adgang til en normal brugers sider.
Hvis man logger ind som admin så får man kun de 3 muligheder som bliver vist i navigations diagrammet. "Add balance", "Delete order" og "log out"
Hvis man derimod logger ind som normal bruger åbnes der en del flere muligheder. Blandt andet til cupcake maker hvor man kan bestille cupcakes og en header som fungere som en navigations bar. Headeren kan man bruge til at navigere sig rundt på siden uanset hvilket sub-side man skulle være inde på. Så hvis man for eksempel er igang med at bestille eller betale for cupcakes, man kan altid vende tilbage til forsiden eller hvor man nu ønsker via headeren.



Særlige forhold

Hvilke informationer gemmer vi i session:

- User gemmes i session, for at holde styr brugeren er logget ind og kunne tilgå attributter.

Hvordan vi har valgt at lave validering af brugerinput:

- Til validering af bruger, har vi en klasse kaldet UserValidator, som ligger i en services package. Dens funktion er at tjekke når brugeren oprettes, om der bliver udfyldt information i felterne, om informationen opfylder specifikke krav, vi har sat op, for at skabe en mere sikker adgangskode.

Hvordan vi har valgt at lave sikkerhed i forbindelse med login:

- Når brugeren logger ind, benytter vi en UserController, hvor vi har bygget en metode der tester at det input, brugeren har tastet matcher noget der findes i databasen, hvis det matcher, så oprettets brugeren som User objekt i systemet og lægger en "sessionAttribute" på brugeren, så der kan indlæses ID,
email og kundens kurv.

Hvilke brugertyper, vi har valgt i databasen og hvordan de er brugt i jdbc:

- I vores database har vi en tabel som vi kalder users. I vores user tabel har vi en kolonne som hedder **role**. En user rolle kan enten være admin eller user. I vores program tjekker vi ved login om en bruger er user eller admin og åbner henholdsvis de respektive HTML sider.

Status på implantation

Vi nåede rigtigt langt i vores projekt og fik implementeret alle user stories. Her er dog en liste over de implantationer vi ikke fik nået:

Aktivitetsdiagram

- I vores projekt indgår der desværre ikke et aktivitetsdiagram. Da dette var det største projekt vi som hold har skulle bygge, valgte vi at fokusere vores tid på implementering af user stories og de diagramtyper vi kendte i forvejen. Aktivitets diagram havde ikke været en del af undervisning op til projektet og derfor følte vi ikke at vi kunne implementere med den givne tidsramme.

Styling

- Dynamisk farver på cupcake i cupcakeMaker når man vælger en specifik top og bund.
- Finishing touches af styling på vores sider.
- Styling især til mobil, mangler der lidt. Men vigtigere er at når man er på mobilversion, virker funktionaliteten til at bestille en cupcake ikke. Planen var at man ville bruge pile, til at skifte top og bund, for at gøre det mere passende til mobilbrugere.

Features

- Password sikkerhed, vi ville gerne have lavet en metode, der gjorde password data, der bliver sendt fra brugeren og til databasen og vice versa til at være krypteret. Vi tænkte at bruge hashcode, men måtte fokusere på at løse User Cases færdigt.
- Resette brugerens kurv efter de bestiller en ordre. Metoden ”payOrder” manglede vi det sidste skridt. Tanken var at tilføje en metode til vores ”BasketMapper”, der slette alle linjer for kundens unikke kurv id, men der var desværre ikke tid.
- Til sidst da vi skulle lave videoen, opdagede vi en bug når man opretter sig som ny bruger, som vi ikke kunne nå at fixe. Det virkede tidligere, men den giver en fejl og opretter ikke en bruger i databasen.

Proces

Hvad var jeres planer for teamets arbejdsform og projektforløbet?

Vi startede projektet ud med at kigge på kundens krav (User Stories), til det havde vi en grundig diskussion og refleksion over hvordan, dette skulle implementeres i programmet. Vigtigt for vores gruppe var også at have en klar aftale og plan for hvad der skulle laves hver dag, til det aftalte vi ikke at kode noget, uden at vi som gruppe havde snakket om det inden. Derfor aftalte vi at de første par dage, ville vi kode sammen og være i et opkald, så vi kunne bygge skelettet sammen.
Efter at have aftalt en gruppekontrakt, begyndte vi at snakke om de diagrammer vi ville have brug for og kom frem til: et ERD diagram og en Domænemodel. Produktet af dette var et godt overblik,
over hvordan projektet ville komme til at se ud.
Som gruppe aftalte vi også at benytte os af GitHub's projects (eller Kanban Board), hvor der kan indsættes små dele af projektets opgaver, som tasks der kan være oprettet, påbegyndt eller færdige.
Vores begrundelse til dette, var at det ville give os et godt overblik over hvad der skulle laves og hvornår. Noget var højere prioritet end andet, derfor var det vigtigt skridt for skridt, at kunne
implementere programmet i små bider.
Noget vi blev hurtigt enige om, var at når vi nåede til at skulle uddelegere tasks til projektet, for at kunne arbejde individuelt, skulle man altid arbejde i en GitHub branch, så der ikke opstod nogle
konflikter.

Hvordan kom det til at forløbe i praksis?

Vi begyndte projektet med en god mavefornemmelse om at kunne blive relativt hurtigt færdige, de første dage af projektet blev brugt sammen i ”Code With Me” featuren i *Intelij*, hvor vi startede med genbruge
hvad vi kunne, fra andre projekter vi har lavet. Vi kunne implementere et stort set fungerende bruger system, hvor der kan oprettes en ny bruger, med en ”UserValidator” klasse, som tager sig af at sikre sig brugeren, opretter sig
med sikre oplysninger. Denne klasse har vi også testet, ved hjælp af en testklasse, der kører alle metoderne igennem og sikrer os, at det fungerer som tiltænkt.
Efter at have implementeret hvad der gav mening fra tidligere projekter, var vores tankegang at vi ville starte med at lave brugerfladen, da dette ville gøre det nemmere for os at senere holde overblik og teste programmets
flow og funktioner, fra en brugers synspunkt. En vigtig erfaring vi gjorde os her, er at man som gruppe bliver nødt til at sætte en begrænsning på tiden brugt på dette.
Man kan hurtigt bruge alt for meget tid, på at sidde og finjustere og dette er som sådan, bedre at have et fuldt fungerende system, der opfylder alle kravene og derefter begynde at finjustere styling,
end det er at have en utrolig flot brugeroverflade, men som ikke opfylder alle kravene, eller har mange fejl og bugs.
Vi havde dog som gruppe, en vigtig samtale, hvor vi besluttede ikke at rette de skæve tekster,

eller forkerte farver, før at vi havde et fuldstændigt fungerende system og dette var en god beslutning, da vi
højst sandsynligt ellers ikke ville have kunnet nå at blive færdige, inden tidsfristen.

Hvad gik godt og hvad kunne have været bedre?

En anden vigtig ting vi har taget med, er hvor meget mere tid det tager at refaktorere en stor rettelse, kontra at bruge lidt ekstra tid på at have et overblik og et opdaterer diagram at kode ud fra. Et gruppemedlem kom med
en ide om at ændre Basket entiteten, hvor der før skulle bruges en liste af Basket for at kunne indeholde al brugerens data, ville vi nu ændre det til at én Basket i stedet indeholdte en liste af Cupcakes og et id.
Dette gav god mening i forhold til processer, og en bruger kunne oprettes i systemet med et Basket id. Men da Basket entiteten allerede var brugt, til at indlæse data til html, på en specifik måde, ved hjælp af en String der indeholdte,
et cupcake navn og pris, var det et enormt arbejde at få refaktoreret systemet.
Hvad der var den "rigtige" og "forkerte" måde, er ikke så vigtigt, som at sikre sig at alle er på samme side, og tid brugt på planlægning er langt mere værdifuldt, end at bare kode løs, for så bagefter at
bruge timevis på at rette og forstå hinandens arbejde.
Med det sagt har vi et rigtig godt samarbejde og tilgang til læring og erfaringer. Vi er alle sammen, nye indenfor programmering og ved derfor at alle fejl. Som team har det også været positivt, for vores resultat at vi alle
har styrker og svagheder. God kommunikation og et mindset, hvor man ikke tager det tungt, hvis det ikke er sin egen ide, der bliver valgt hver gang til at løse en task. Konstruktiv kritik og at kunne acceptere andre meninger end sig egen,
har også været vigtigt.

Hvad har I lært af processen og hvad vil I evt. gøre anderledes næste gang?

Til næste projekt mener vi, at det vil gavne vores arbejdsgang at bruge, endnu mere tid på at lave diagrammer og holde dem opdaterede. Til dette mener vi også, det vil give os et bedre resultat at bruge mindre tid på design og mere tid på
systemudvikling. Fremadrettet er vi højst sandsynligt også nødsaget, til at bruge mindre tid på at kode sammen, da det i princippet er en mindre effektiv fremgangsmetode, selvfølgelig er der styrker i det, men tre mennesker, der kigger på det
samme, resultere i en i hver fald, langsommere proces. Vi har som gruppe, fået en bedre forståelse for GitHub og at arbejde i branches, med hver sit fokusområde, resultere i en højere effektivitet.
Konklusionen er at med en robust plan, som kan justeres i takt med systemet, laves og bliver mere komplekst, er en rigtig god fremgangsmåde. At man kan kommunikere frit og ofte som gruppe, gør at alle er på samme side og fejl og mangler, kan hurtigt
rettes og udfyldes og et solidt resultat kan bedre opnås.